AULA VIRTUALE DEL 2026-03-20

LABORATORIO DI INTELLIGENZA ARTIFICIALE

ESERCIZIO PROPOSTO

Realizza un prototipo semplificato dell'assistente che includa le seguenti funzionalità.

1. Classificazione del testo

Utilizza un modello supervisionato (es. Logistic Regression o Transformer pre-addestrato).

Classifica ogni documento in una delle seguenti categorie:

informativo

richiesta_urgente

reclamo

È sufficiente un dataset di esempio o dati simulati.

2. Generazione di una risposta testuale

Utilizza un modello Transformer pre-addestrato (es. Hugging Face).

Genera:

una sintesi del documento, oppure

una risposta automatica adeguata alla categoria individuata.

Mostra come una variazione del prompt (es. più dettagliato vs generico) influenzi l'output.

3. Simulazione di comportamento agentico

In base alla categoria del testo:

se richiesta_urgente → simula l'apertura di un ticket prioritario;

se reclamo → simula l'inoltro al reparto competente;

se informativo → archivia il documento.

Implementa l'azione come funzione Python, chiarendo la logica decisionale dell'agente.

Soluzione proposta

1. Obiettivo del prototipo

L'azienda richiede un assistente intelligente capace di:

1. **analizzare documenti testuali brevi**;
2. **classificarli** in categorie semantiche;
3. **generare una sintesi o una risposta automatica** in linguaggio naturale;
4. **simulare un comportamento agentico**, cioè eseguire un'azione diversa in funzione del contenuto.

Dal punto di vista del corso, il problema integra tre blocchi:

- **IA discriminativa**: classificazione supervisionata;
- **IA generativa**: generazione di testo con Transformer;
- **IA agentica**: scelta ed esecuzione di un'azione sulla base dell'output del modello.

2. Architettura logica del sistema

Il prototipo può essere descritto con questa pipeline:

- **Documento testuale**
- **Preprocessing leggero**
- **Classificatore supervisionato**
- **Etichetta di classe** (informativo, richiesta_urgente, reclamo)
- **Modulo generativo Transformer**
- **Sintesi oppure risposta automatica**
- **Agente software**
- **Azione simulata** (archivia, apri ticket prioritario, inoltra al reparto competente)

Questa ipotesi di scomposizione del problema in sottoproblemi può essere fatta perché separa:

- il problema di **comprensione e decisione**;
- il problema di **generazione linguistica**;
- il problema di **esecuzione dell'azione**.

3. Dataset di esempio

Poiché il testo dell'esercizio specifica che è sufficiente un **dataset simulato**, costruiamo un piccolo insieme etichettato (generazione di dati sintetici).

Esempi di dati

```
dataset = [
  # informativo
  ("Vi informiamo che il servizio sarà aggiornato domenica dalle 2 alle 4.", "informativo"),
  ("Il contratto è stato rinnovato e allego la documentazione aggiornata.", "informativo"),
  ("Desidero sapere che il pagamento risulta correttamente registrato.", "informativo"),
  ("Comunichiamo che il nuovo listino entrerà in vigore dal prossimo mese.", "informativo"),

  # richiesta_urgente
  ("Il server è irraggiungibile, serve un intervento immediato.", "richiesta_urgente"),
  ("Ho bisogno di assistenza subito, non riesco ad accedere al portale.", "richiesta_urgente"),
  ("Emergenza: il sistema di prenotazione è bloccato da un'ora.", "richiesta_urgente"),
  ("Per favore intervenite con priorità alta, il servizio è fermo.", "richiesta_urgente"),

  # reclamo
  ("Segnalo un grave disservizio e sono molto insoddisfatto del supporto ricevuto.", "reclamo"),
  ("Il prodotto acquistato non funziona e desidero presentare un reclamo.", "reclamo"),
  ("Ho già scritto due volte senza ottenere risposta, servizio pessimo.", "reclamo"),
  ("Sono deluso dalla gestione della mia richiesta e chiedo spiegazioni.", "reclamo"),
]
```

Dal punto di vista accademico, è bene osservare che questo dataset è:

- **piccolo**;
- **sbilanciato rispetto al mondo reale**;
- utile **solo come prototipo dimostrativo**.

In un progetto di tipo “entreprise” servirebbero più esempi, validazione rigorosa, metriche più solide e probabilmente un fine-tuning o un dataset interno.

4. Classificazione del testo

L'esercizio chiede un modello supervisionato, ad esempio:

- **Logistic Regression**, soluzione semplice e coerente con un laboratorio introduttivo;
- oppure un **Transformer pre-addestrato** per text classification.

Qui propongo come soluzione principale **TF-IDF + Logistic Regression**, perché:

1. è semplice da addestrare;
2. è interpretabile;
3. è adatta a dataset piccoli;
4. rispetta pienamente la richiesta di "modello supervisionato".

Implementazione

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

texts = [x[0] for x in dataset]
labels = [x[1] for x in dataset]

X_train, X_test, y_train, y_test = train_test_split(
   texts, labels, test_size=0.25, random_state=42, stratify=labels
)

clf = Pipeline([
   ("tfidf", TfidfVectorizer(lowercase=True, ngram_range=(1,2))),
   ("model", LogisticRegression(max_iter=1000))
])

clf.fit(X_train, y_train)
```

```
pred = clf.predict(X_test)
print(classification_report(y_test, pred))
```

Commento metodologico

La pipeline usa:

- **TF-IDF** per rappresentare i documenti in forma numerica;
- **Logistic Regression** come classificatore multiclasse.

Questa scelta è appropriata per testi brevi come email, ticket e richieste clienti, perché le parole chiave e le loro combinazioni contengono spesso abbastanza informazione discriminativa:

- “urgente”, “subito”, “immediato” → richiesta_urgente
- “insoddisfatto”, “reclamo”, “pessimo” → reclamo
- “informiamo”, “comunichiamo”, “allego” → informativo

Funzione di classificazione

```
def classify_document(text, classifier):
    label = classifier.predict([text])[0]
    probs = classifier.predict_proba([text])[0]
    classes = classifier.classes_
    confidence = {classes[i]: float(probs[i]) for i in range(len(classes))}
    return label, confidence
```

Esempio:

```
doc = "Il servizio è bloccato, vi chiedo un intervento immediato."
label, confidence = classify_document(doc, clf)
print(label)
print(confidence)
```

Output atteso, ad esempio:

```
richiesta_urgente
{'informativo': 0.08, 'reclamo': 0.12, 'richiesta_urgente': 0.80}
```

5. Generazione di una risposta testuale con Transformer

L'esercizio richiede un **modello Transformer pre-addestrato**, ad esempio tramite Hugging Face.

Ci sono due possibilità corrette:

1. generare una **sintesi**;
2. generare una **risposta automatica** coerente con la categoria.

Per essere pienamente aderenti alla richiesta, mostro entrambe le idee, ma implemento la seconda in modo più esplicito.

5.1 Esempio con modello Hugging Face

Per un prototipo didattico possiamo usare una pipeline text2text-generation.

```
from transformers import pipeline


generator = pipeline(
   "text2text-generation",
   model="google/flan-t5-base"
)
```

5.2 Generazione di sintesi

```
def summarize_document(text):
   prompt = f"Riassumi in modo chiaro e conciso il seguente testo:\n\n{text}"
   result = generator(prompt, max_new_tokens=60, do_sample=False)
   return result[0]["generated_text"]
```

Esempio:

```
doc = "Il contratto è stato aggiornato e la nuova documentazione sarà inviata a tutti i clienti entro venerdì."

print(summarize_document(doc))
```

Possibile output:

Il testo informa che il contratto è stato aggiornato e che la documentazione sarà inviata ai clienti entro venerdì.

5.3 Generazione di risposta automatica in base alla categoria

Qui usiamo il risultato della classificazione per guidare il prompt.

```
def generate_response(text, label, detailed=False):
  if detailed:
    prompt = f"""
Sei un assistente clienti aziendale.
Categoria del messaggio: {label}

Testo del cliente:
{text}

Scrivi una risposta professionale, cortese e coerente con la categoria.
- Se è una richiesta urgente, comunica presa in carico immediata e priorità alta.
- Se è un reclamo, mostra empatia e informa dell'inoltro al reparto competente.
- Se è informativo, conferma ricezione e archiviazione.
Mantieni un tono formale e chiaro.
"""
  else:
    prompt = f"Scrivi una risposta al seguente messaggio, categoria {label}: {text}"

  result = generator(prompt, max_new_tokens=80, do_sample=False)
  return result[0]["generated_text"]
```

6. Effetto della variazione del prompt

Questa parte è importante, perché il testo dell'esercizio chiede esplicitamente di mostrare come un **prompt generico** e un **prompt più dettagliato** producano output diversi (...volendo applicando anche specifiche tecniche di prompt engineering).

Supponiamo di avere questo documento:

doc = "Ho già scritto due volte senza ricevere risposta. Sono molto insoddisfatto del servizio."

label, _ = classify_document(doc, clf)

Prompt generico

print(generate_response(doc, label, detailed=False))

Possibile output:

Ci dispiace per il disagio. La sua segnalazione sarà presa in carico.

Prompt dettagliato

print(generate_response(doc, label, detailed=True))

Possibile output:

Gentile cliente, ci scusiamo sinceramente per il disservizio e per la mancata risposta alle precedenti comunicazioni. Abbiamo preso in carico il suo reclamo e lo stiamo inoltrando al reparto competente per una verifica prioritaria. Riceverà un aggiornamento non appena possibile.

Osservazione critica

Dal punto di vista scientifico, la differenza è chiara:

- il **prompt generico** lascia maggiore libertà al modello e produce una risposta più breve e meno controllata;
- il **prompt dettagliato** impone:
    - tono,
    - obiettivo,
    - vincoli semantici,
    - comportamento atteso.

Quindi, in un sistema reale, la **prompt engineering** è uno strumento di controllo importante della generazione.

7. Simulazione del comportamento agentico

Qui entriamo nel terzo modulo logico: il sistema non si limita a classificare e generare testo, ma **agisce**.

L'agente implementa una politica decisionale semplice:

- richiesta_urgente → apri ticket prioritario;
- reclamo → inoltra al reparto competente;
- informativo → archivia documento.

Questa è una forma elementare ma corretta di **IA agentica**, perché c'è:

1. percezione dell'input;
2. interpretazione;
3. decisione;
4. esecuzione di un'azione.

7.1 Funzioni Python per le azioni

```
def open_priority_ticket(text):
  return {
    "azione": "apertura_ticket_prioritario",
    "stato": "eseguito",
    "dettaglio": f"Ticket prioritario aperto per il testo: {text}"
  }


def forward_to_department(text):
  return {
    "azione": "inoltro_reparto_competente",
    "stato": "eseguito",
    "dettaglio": f"Documento inoltrato al reparto reclami: {text}"
  }


def archive_document(text):
```

```
    return {
        "azione": "archiviazione",
        "stato": "eseguito",
        "dettaglio": f"Documento archiviato: {text}"
    }
```

7.2 Logica decisionale dell’agente

```
def agent_action(text, label):
    if label == "richiesta_urgente":
        return open_priority_ticket(text)
    elif label == "reclamo":
        return forward_to_department(text)
    elif label == "informativo":
        return archive_document(text)
    else:
        return {
            "azione": "nessuna",
            "stato": "errore",
            "dettaglio": "Categoria non riconosciuta"
        }
```

Spiegazione soluzione

Questa è una politica **rule-based** guidata dalla classe predetta dal modello.
È una scelta corretta in un prototipo, perché consente di separare:

- la parte statistica: classificazione del testo;
- la parte simbolica/decisionale: regole dell'agente.

In una versione più avanzata si potrebbero introdurre:

- soglie di confidenza;
- gestione dell'incertezza;
- multi-step reasoning;
- chiamate API reali al sistema di ticketing.

8. Pipeline completa end-to-end

Adesso uniamo tutti i moduli.

```
def intelligent_assistant(document, classifier, detailed_prompt=True):
  # 1. classificazione
  label, confidence = classify_document(document, classifier)

  # 2. generazione risposta
  response = generate_response(document, label, detailed=detailed_prompt)

  # 3. azione agentica
  action_result = agent_action(document, label)

  return {
    "documento": document,
    "categoria_predetta": label,
    "confidenza": confidence,
    "risposta_generata": response,
    "azione_agente": action_result
```

```
    }
```

Esempio di esecuzione

```
doc1 = "Il server aziendale non è raggiungibile, intervenite subito."

result = intelligent_assistant(doc1, clf, detailed_prompt=True)


for k, v in result.items():
    print(f"{k}: {v}")
```

Output atteso

documento: Il server aziendale non è raggiungibile, intervenite subito.

categoria_predetta: richiesta_urgente

confidenza: {'informativo': 0.05, 'reclamo': 0.10, 'richiesta_urgente': 0.85}

risposta_generata: Gentile cliente, abbiamo preso in carico la sua segnalazione con priorità alta...

azione_agente: {'azione': 'apertura_ticket_prioritario', 'stato': 'eseguito', 'dettaglio': 'Ticket prioritario aperto...'}

9. Versione alternativa: classificatore Transformer

Se si volesse rendere il prototipo ancora più vicino ai moderni sistemi NLP, si potrebbe sostituire Logistic Regression con un Transformer di classificazione, ad esempio BERT fine-tuned.

Schema concettuale:

```
from transformers import pipeline


classifier = pipeline(
    "text-classification",
    model="distilbert-base-uncased-finetuned-sst-2-english"
)
```

Tuttavia, per il problema specifico questo non è immediatamente pronto, perché il modello dovrebbe essere **riadattato alle tre etichette del task** (informativo, richiesta_urgente,

reclamo).
Per un esame, dunque, la soluzione **TF-IDF + Logistic Regression** è in realtà più rigorosa, perché è **davvero addestrabile sul dataset simulato** richiesto dalla traccia.

10. Discussione critica della soluzione

Punti di forza

- architettura modulare e chiara;
- uso di un classificatore supervisionato conforme alla traccia;
- uso di Transformer pre-addestrato per la generazione;
- simulazione esplicita del comportamento agentico;
- possibilità di mostrare l'effetto del prompt.

Limiti

- dataset troppo piccolo per generalizzare;
- assenza di vera validazione robusta;
- possibili errori di classificazione su testi ambigui;
- generazione non sempre controllabile al 100%;
- agente basato su regole semplici, non su pianificazione complessa.

Miglioramenti futuri

- aumentare il dataset e bilanciarlo;
- introdurre cross-validation;
- fine-tuning di un modello Transformer sulle classi specifiche;
- uso di RAG o base di conoscenza aziendale;
- integrazione con sistemi reali di ticketing e CRM.

11. Conclusione finale

Il prototipo realizzato soddisfa i tre requisiti fondamentali dell'esercizio: classificazione supervisionata del testo, generazione di output linguistico tramite Transformer pre-addestrato e simulazione di comportamento agentico. La soluzione è coerente con gli obiettivi del corso, in quanto integra metodologie di IA discriminativa, NLP generativo e logica decisionale agentica in un'unica pipeline software. In un contesto reale il sistema richiederebbe dataset più ampi, validazione sperimentale più rigorosa e integrazione con servizi aziendali effettivi, ma come prova di laboratorio rappresenta un'implementazione corretta, modulare ed estendibile.

12. Codice completo compatto

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from transformers import pipeline

# ========================
# 1. Dataset simulato
# ========================
dataset = [
    ("Vi informiamo che il servizio sarà aggiornato domenica dalle 2 alle 4.", "informativo"),
    ("Il contratto è stato rinnovato e allego la documentazione aggiornata.", "informativo"),
    ("Desidero comunicare che il pagamento risulta correttamente registrato.", "informativo"),
    ("Comunichiamo che il nuovo listino entrerà in vigore dal prossimo mese.", "informativo"),

    ("Il server è irraggiungibile, serve un intervento immediato.", "richiesta_urgente"),
    ("Ho bisogno di assistenza subito, non riesco ad accedere al portale.", "richiesta_urgente"),
    ("Emergenza: il sistema di prenotazione è bloccato da un'ora.", "richiesta_urgente"),
    ("Per favore intervenite con priorità alta, il servizio è fermo.", "richiesta_urgente"),
```

```
    ("Segnalo un grave disservizio e sono molto insoddisfatto del supporto ricevuto.", "reclamo"),
    ("Il prodotto acquistato non funziona e desidero presentare un reclamo.", "reclamo"),
    ("Ho già scritto due volte senza ottenere risposta, servizio pessimo.", "reclamo"),
    ("Sono deluso dalla gestione della mia richiesta e chiedo spiegazioni.", "reclamo"),
]

texts = [x[0] for x in dataset]
labels = [x[1] for x in dataset]

# =========================
# 2. Classificatore supervisionato
# =========================
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.25, random_state=42, stratify=labels
)

clf = Pipeline([
    ("tfidf", TfidfVectorizer(lowercase=True, ngram_range=(1,2))),
    ("model", LogisticRegression(max_iter=1000))
])

clf.fit(X_train, y_train)

def classify_document(text, classifier):
    label = classifier.predict([text])[0]
    probs = classifier.predict_proba([text])[0]
    classes = classifier.classes_
```

```
    confidence = {classes[i]: float(probs[i]) for i in range(len(classes))}
    return label, confidence

# ========================
# 3. Modello generativo Transformer
# ========================
generator = pipeline("text2text-generation", model="google/flan-t5-base")

def summarize_document(text):
    prompt = f"Riassumi in modo chiaro e conciso il seguente testo:\n\n{text}"
    result = generator(prompt, max_new_tokens=60, do_sample=False)
    return result[0]["generated_text"]

def generate_response(text, label, detailed=False):
    if detailed:
        prompt = f"""
Sei un assistente clienti aziendale.
Categoria del messaggio: {label}

Testo:
{text}

Scrivi una risposta professionale, cortese e coerente con la categoria.
- Se è una richiesta urgente, comunica presa in carico immediata e priorità alta.
- Se è un reclamo, mostra empatia e informa dell'inoltro al reparto competente.
- Se è informativo, conferma ricezione e archiviazione.
"""
    else:
        prompt = f"Scrivi una risposta al seguente messaggio, categoria {label}: {text}"
```

```
    result = generator(prompt, max_new_tokens=80, do_sample=False)
    return result[0]["generated_text"]

# ========================
# 4. Azioni agentiche
# ========================
def open_priority_ticket(text):
    return {
        "azione": "apertura_ticket_prioritario",
        "stato": "eseguito",
        "dettaglio": f"Ticket prioritario aperto per: {text}"
    }

def forward_to_department(text):
    return {
        "azione": "inoltro_reparto_competente",
        "stato": "eseguito",
        "dettaglio": f"Documento inoltrato al reparto reclami: {text}"
    }

def archive_document(text):
    return {
        "azione": "archiviazione",
        "stato": "eseguito",
        "dettaglio": f"Documento archiviato: {text}"
    }

def agent_action(text, label):
```

```
    if label == "richiesta_urgente":
        return open_priority_ticket(text)
    elif label == "reclamo":
        return forward_to_department(text)
    elif label == "informativo":
        return archive_document(text)
    else:
        return {
            "azione": "nessuna",
            "stato": "errore",
            "dettaglio": "Categoria non riconosciuta"
        }

# ========================
# 5. Pipeline completa
# ========================
def intelligent_assistant(document, classifier, detailed_prompt=True):
    label, confidence = classify_document(document, classifier)
    response = generate_response(document, label, detailed=detailed_prompt)
    action_result = agent_action(document, label)

    return {
        "documento": document,
        "categoria_predetta": label,
        "confidenza": confidence,
        "risposta_generata": response,
        "azione_agente": action_result
    }
```

```
# ========================
# 6. Esempio d'uso
# ========================
doc = "Il server aziendale non è raggiungibile, intervenite subito."
result = intelligent_assistant(doc, clf, detailed_prompt=True)

print(result)
print("\nPrompt generico:")
print(generate_response(doc, result["categoria_predetta"], detailed=False))
print("\nPrompt dettagliato:")
print(generate_response(doc, result["categoria_predetta"], detailed=True))
```